Task-11:- Documentation(Deploy a 2-Tier Application on AWS -Bookstore)

1. Introduction

1.1 Purpose

The purpose of this technical document is to describe the deployment of the **Bookstore API application** on AWS using a **2-tier architecture** across **multiple Availability Zones**.

The deployment separates the **application layer** and **database layer** to ensure better reliability, scalability, and security.

This document is intended for:

- CTO
- DevOps Engineers
- Developers
- Solution Architects
- Infrastructure Engineers
- Resellers

1.2 Scope

This document describes the deployment of the Bookstore API application using DevOps practices on AWS infrastructure.

The scope includes:

- Deployment of FastAPI application on **Amazon EC2**
- Database setup using **MongoDB Atlas**
- Multi-Availability Zone architecture implementation
- Secure communication between application and database
- API testing and verification

---

2. Overview of DevOps Project

2.1 Definition

DevOps combines development and operations practices to enable faster application delivery, improved infrastructure management, and better collaboration between development and

operations teams.

In this project, DevOps principles were applied to deploy the **Bookstore API application on AWS** using a **2-tier architecture with high availability across multiple Availability Zones**.

Benefits include:

- Improved reliability
- Better fault tolerance
- Scalable infrastructure
- Secure application deployment

2.2 Key Principles (AWS Well-Architected Framework)

The solution aligns with the **6 pillars of the AWS Well-Architected Framework**.

Operational Excellence

Automation of deployment steps ensures consistent infrastructure management.

Security

Database credentials are stored in environment variables and database access is restricted.

Reliability

Application infrastructure spans **multiple Availability Zones**, improving resilience.

Performance Efficiency

FastAPI ensures efficient API performance with lightweight execution.

Cost Optimization

Free-tier EC2 instances and MongoDB Atlas free cluster reduce operational costs.

Sustainability

Minimal infrastructure resources reduce environmental impact.

2.3 Goals and Objectives

The objectives of this project include:

- Deploy the Bookstore API application on AWS
- Implement **2-tier architecture**

- Deploy infrastructure across **multiple Availability Zones**
- Separate application and database layers
- Securely integrate application and database
- Validate API functionality through testing

2.4 Constraints and Assumptions

Constraints

- Limited infrastructure budget
- Deployment must be completed within **3 hours**
- Free-tier services preferred

Assumptions

- AWS account access available
- MongoDB Atlas account created
- Internet access available for EC2 instance

---

3. Architecture and Infrastructure

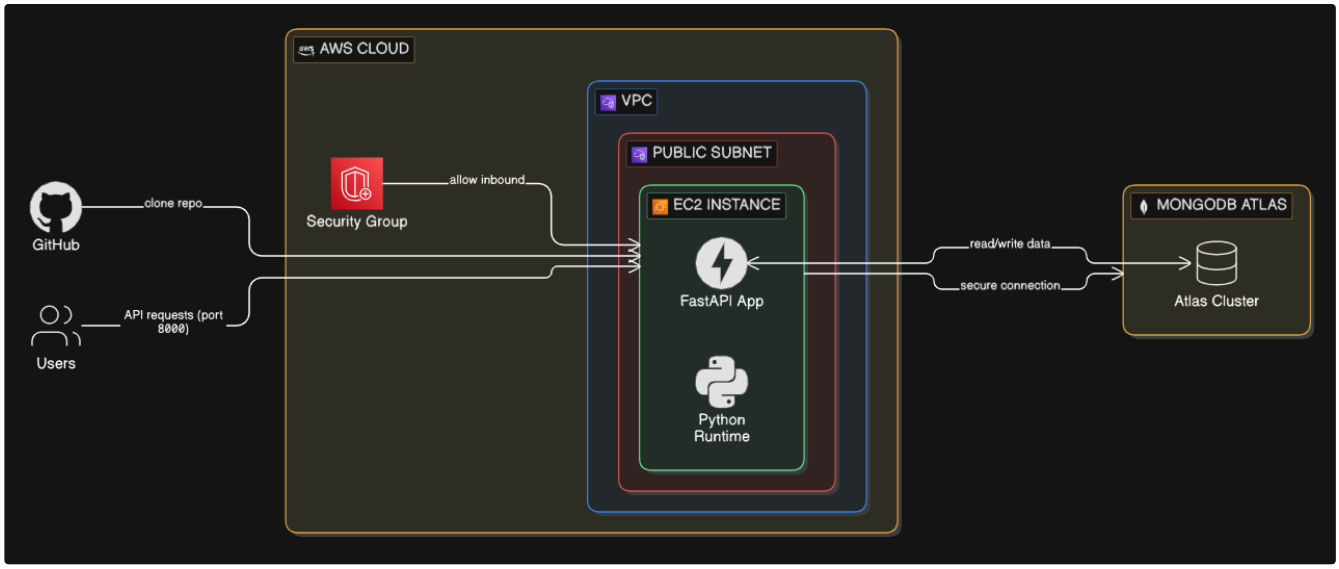
3.1 Infrastructure Overview

The infrastructure includes:

- EC2 instance hosting the FastAPI application
- MongoDB Atlas hosting the database
- VPC networking with security groups
- Deployment across multiple Availability Zones

Architecture Overview

```
User
 │
 │ HTTP Request
 ▼
Application Layer
EC2 Instance (AZ-1)
 │
 │ Secure Database Connection
 ▼
Database Layer
MongoDB Atlas Cluster
```



3.2 Environments

The following environments were considered in this deployment:

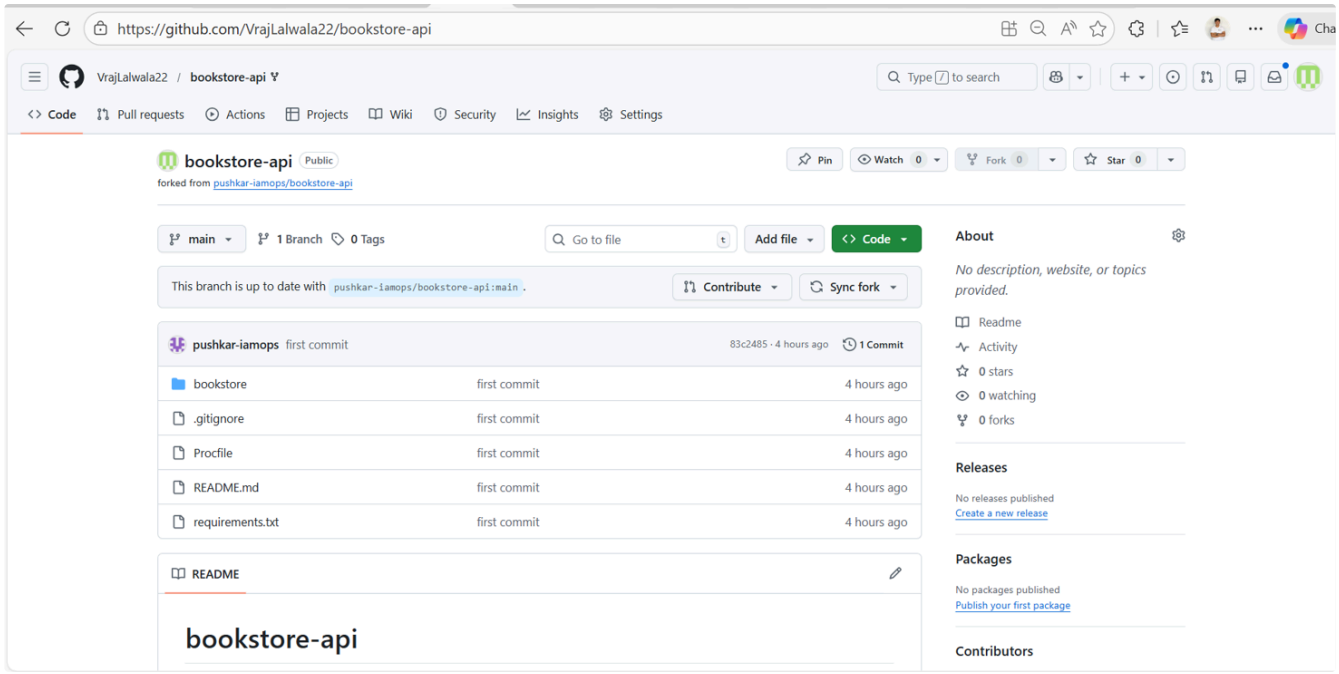
Environment	Purpose
Development	Code development
Testing	API functionality verification
Production (Future)	Final deployment environment

This task focuses mainly on the **development/testing environment**.

3.3 Tools and Technologies

Tool	Purpose
AWS EC2	Application hosting
MongoDB Atlas	Managed database service
FastAPI	Backend API framework
Python	Application runtime
GitHub	Source code repository
Uvicorn	ASGI web server

SSH	Secure server access

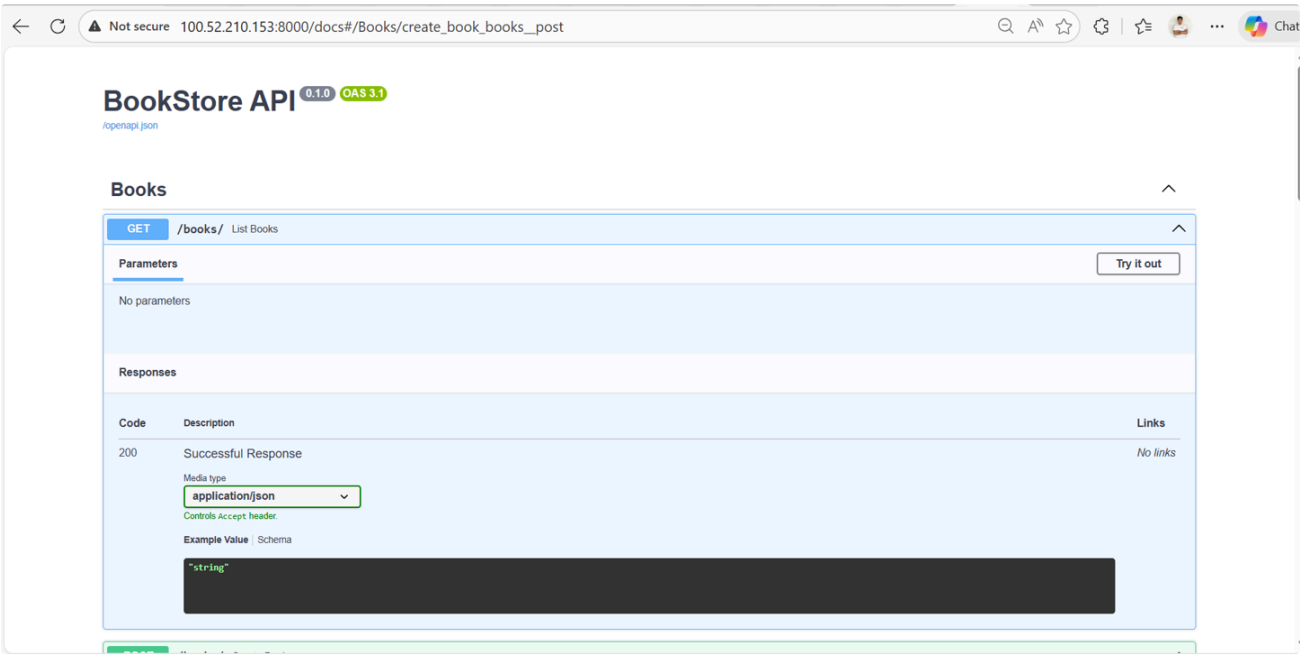


3.4 Application Architecture & User Request Flow

Application Flow

```
User Request
      |
      ▼
Internet Gateway
      |
      ▼
EC2 Application Server (Availability Zone-1)
      |
      ▼
MongoDB Atlas Database Cluster
```

1. User sends API request
2. Request reaches FastAPI server
3. Application processes request
4. Data stored/retrieved from MongoDB
5. Response returned to user



---

4. Development Practices

4.1 Version Control

The project source code is maintained in a GitHub repository.

Repository used:

```
https://github.com/pushkar-iamops/bookstore-api
```

Git was used for:

- Code versioning
- Source control management
- Deployment cloning

4.2 Continuous Integration

For this project CI was simulated using manual deployment steps:

1. Code stored in GitHub
2. Repository cloned on EC2
3. Dependencies installed
4. Application executed

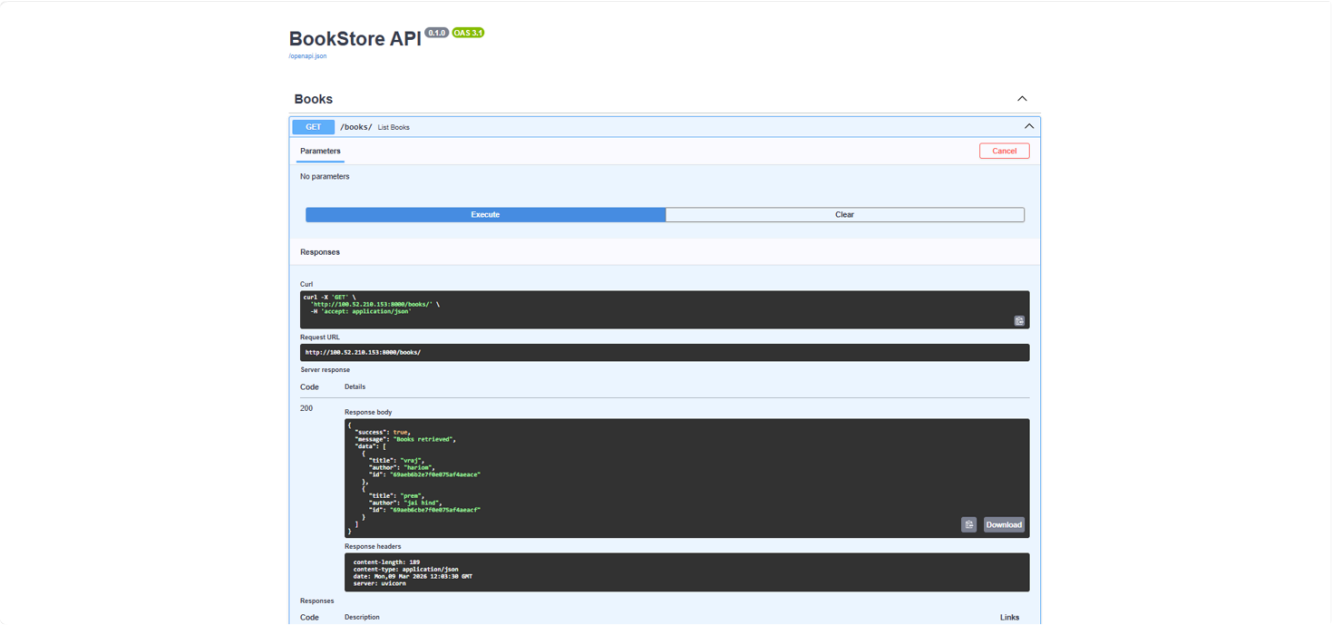
4.3 Automated Testing

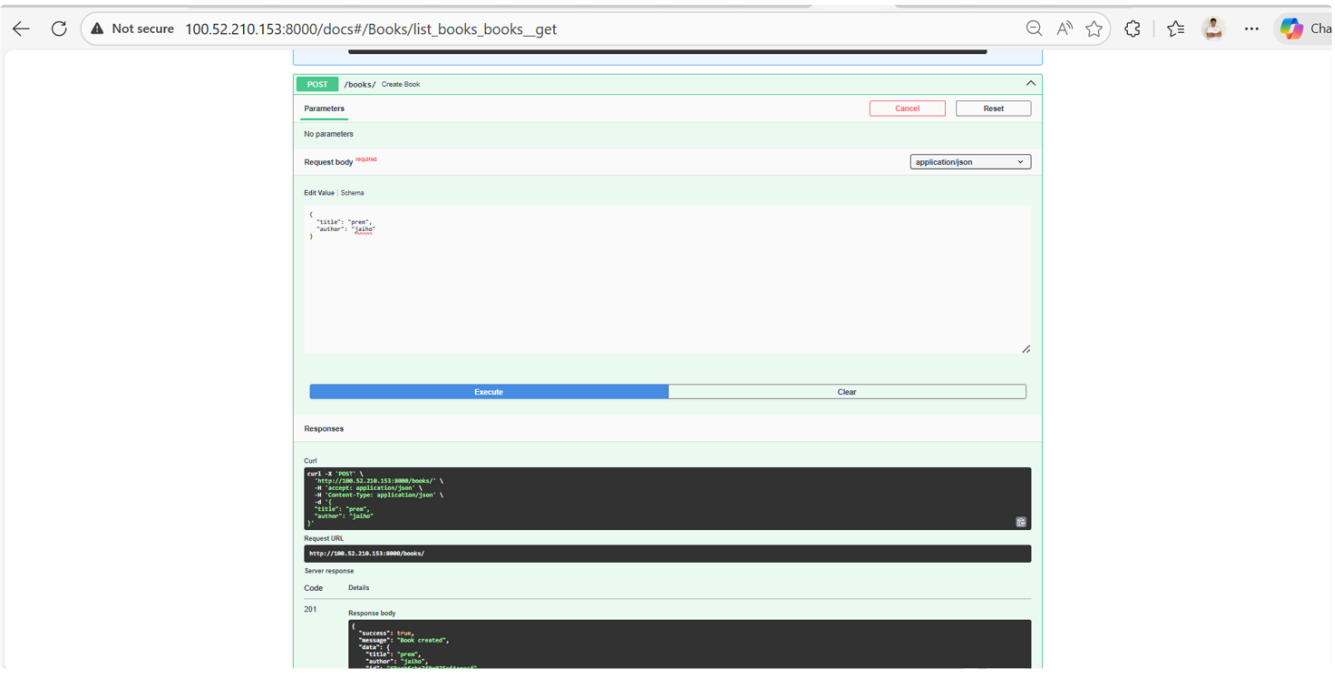
API endpoints were tested using:

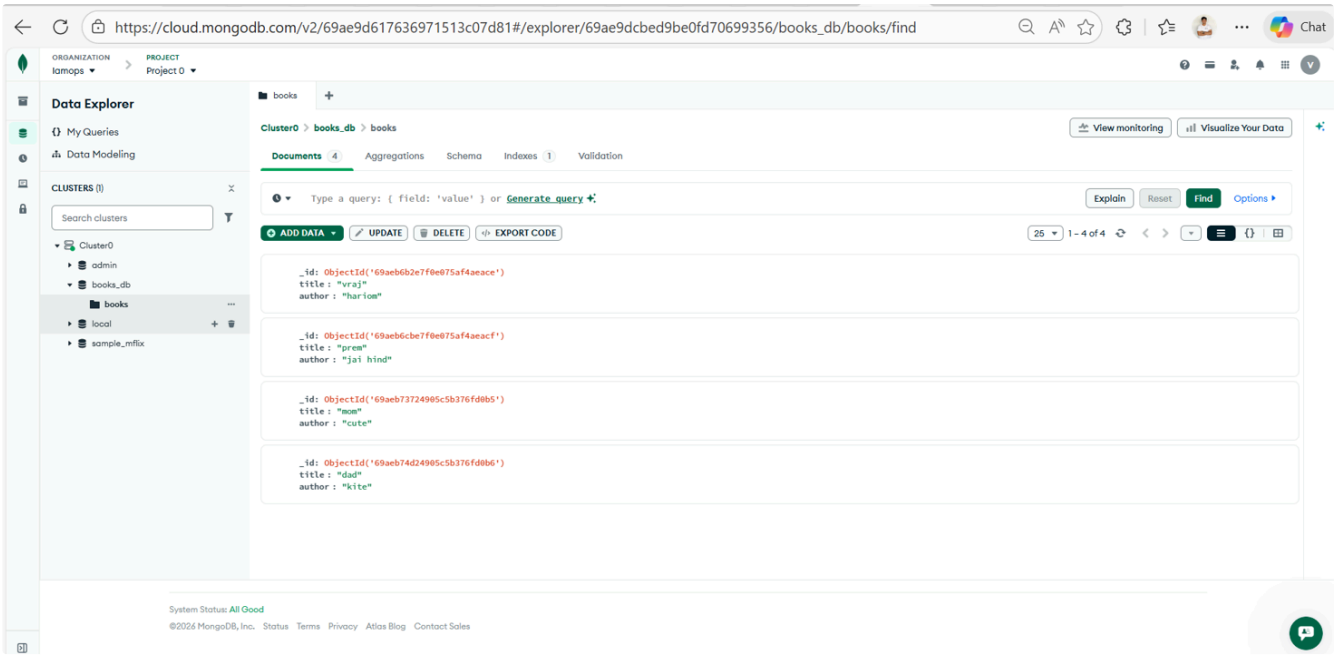
- FastAPI Swagger UI
- HTTP request testing

Example endpoint tested:

```
POST /books
GET /books
```







4.4 Code Quality and Analysis

The project follows good coding practices including:

- Modular API routes
- Environment variable configuration
- Separation of database and application logic

---

5. Deployment and Release Management

5.1 Deployment Strategy

The application deployment process followed these steps:

1. Launch EC2 instance in Availability Zone-1
2. Connect via SSH
3. Install Python dependencies
4. Clone GitHub repository
5. Configure environment variables
6. Run FastAPI server using Uvicorn

Example command:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

```
(venv) ubuntu@ip-10-1-1-110:~/bookstore-api$ cd bookstore/
(venv) ubuntu@ip-10-1-1-110:~/bookstore-api/bookstore$ export MONGO_URI="mongodb+srv://bookstore_user:StrongPassword@cluster0.ut8gym1
.mongodb.net/?appName=Cluster0"
(venv) ubuntu@ip-10-1-1-110:~/bookstore-api/bookstore$ uvicorn main:app --host 0.0.0.0 --port 8000
INFO:     Started server process [4092]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     27.116.48.171:58986 - "GET /docs HTTP/1.1" 200 OK
INFO:     27.116.48.171:58986 - "GET /openapi.json HTTP/1.1" 200 OK
```

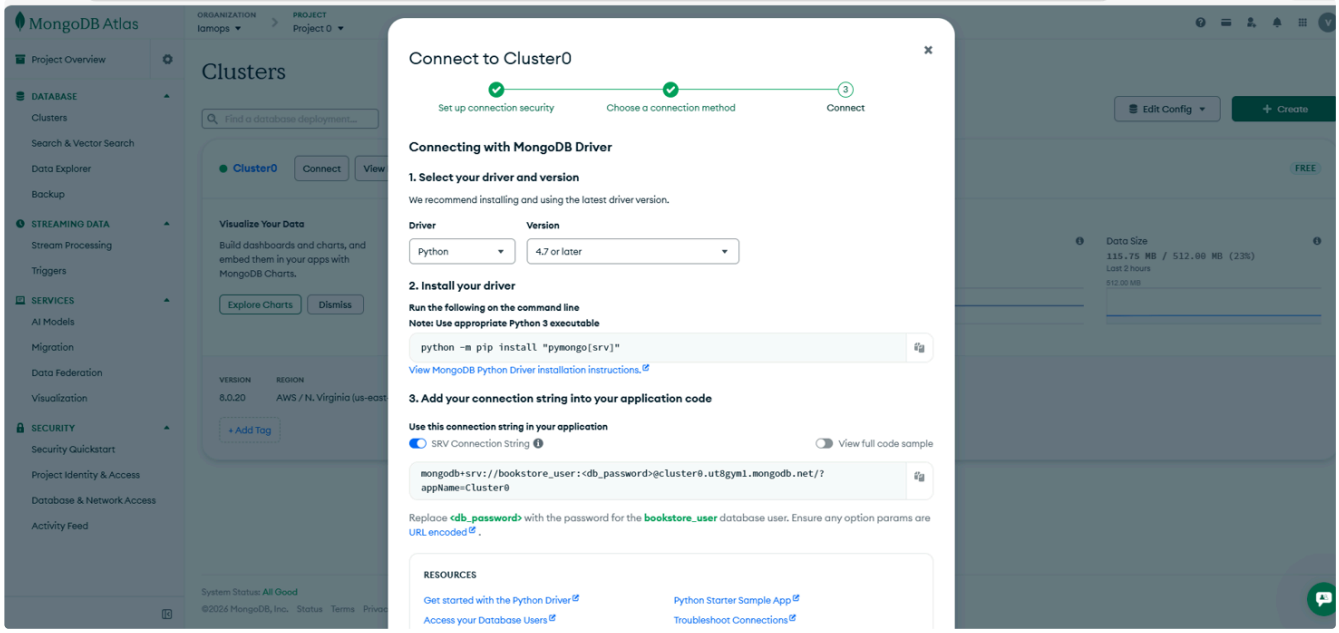
5.2 Configuration Management

Configuration values such as database connection strings are stored in environment variables.

Example `.env` file:

```
MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/bookstore
```

This ensures credentials are not exposed in the source code.



5.3 Continuous Delivery / Deployment

The deployment pipeline consists of:

```
GitHub Repository → EC2 Server → Application Execution
```

Future improvements may include automated CI/CD pipelines.

---

6. Monitoring and Incident Management

6.1 Monitoring Tools and Metrics

Monitoring methods used include:

- EC2 instance monitoring
- Application logs
- API response monitoring

6.2 Alerting and Notification

Alerts can be configured using AWS monitoring services for:

- Instance failure
- Application downtime
- Performance issues

6.3 Incident Management

Incident response includes:

1. Identify issue
2. Check server logs
3. Restart application service
4. Verify database connectivity

---

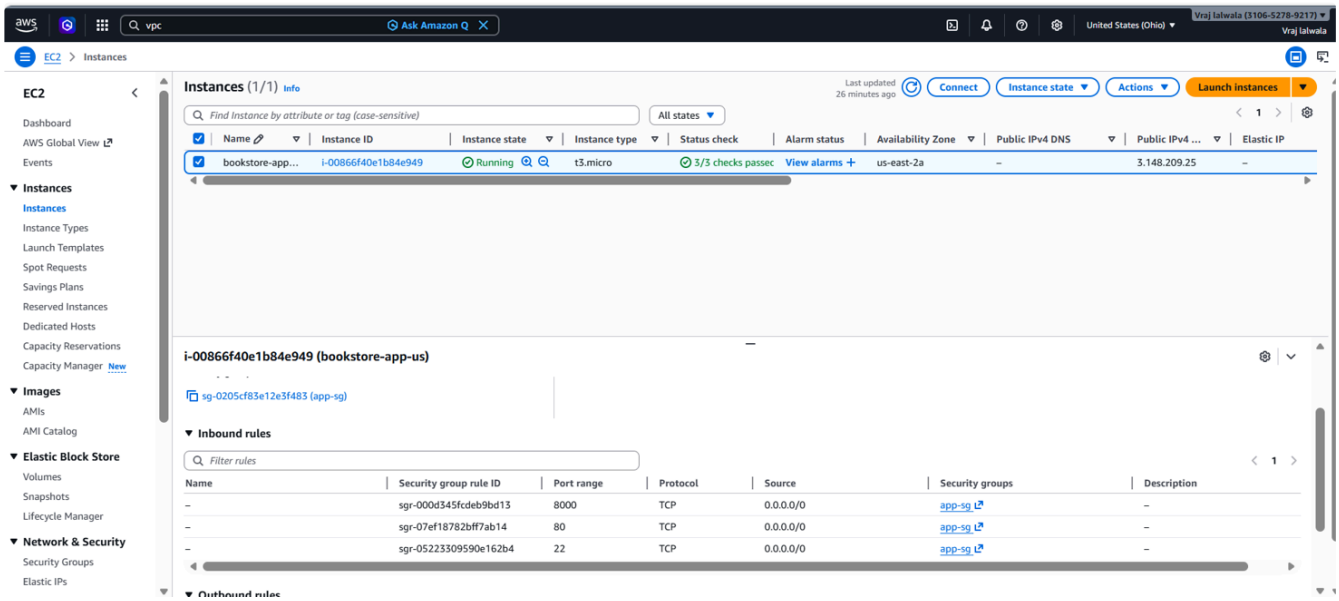
7. Security & Compliance

Security practices implemented:

- Database credentials stored in environment variables
- MongoDB network access restrictions
- EC2 security group rules

Security group configuration:

Port	Purpose
22	SSH access
8000	FastAPI API



8. Cost Optimization

To minimize infrastructure cost, the following strategies were used:

- AWS Free Tier EC2 instance
- MongoDB Atlas free cluster
- Minimal infrastructure usage

Cost estimation includes:

Service	Cost
EC2 t2.micro	Free Tier
MongoDB Atlas	Free Tier

This ensures minimal operational cost during development.

10. Implementation Timeline

10.1 Milestones and Deliverables

Phase	Description
Phase 1	Infrastructure setup

Phase 2	Application deployment
Phase 3	Database configuration
Phase 4	API testing

---

11. Continuous Improvement

11.1 Feedback and Retrospectives

Feedback from developers and DevOps engineers helps improve deployment processes and infrastructure reliability.

11.2 Iterative Enhancements

Future improvements may include:

- CI/CD pipeline automation
- Containerization using Docker
- Infrastructure automation using Terraform

---

12. Conclusion

12.1 Summary

The Bookstore API application was successfully deployed on AWS using a **2-tier architecture**.

The deployment included:

- FastAPI application hosted on EC2
- MongoDB Atlas database configuration
- Secure API-database connectivity
- Successful API functionality verification

This architecture provides a scalable and secure environment for application deployment.

12.2 Next Steps

Future enhancements include:

- CI/CD pipeline integration
- Load balancing
- Containerized deployment
- Infrastructure as Code implementation

---

13. Customer Acknowledgement

Name:

Title:

Signature:

Date: